

# CS 2053M 计算机系统 III

## 24-25 春夏学期期末回忆卷

整理自 CC98 论坛 + 考场抄写，题目不全，仅供参考

### 一、选择题（共 15 道）

1. 在没有前递 (forwarding)、分支预测，且指令内存和数据内存分开的五级流水线中运行下列指令。在 I5 之前，发生了哪种或哪些 Hazard?

```
I1: LW    x1, 0(x2)
I2: ADD   x3, x1, x4
I3: SUB   x5, x3, x6
I4: BEQ   x3, x0, L1
I5: ADDI  x7, x7, 1
```

**解：**RAW hazard:  $I1 \rightarrow I2$  (x1),  $I2 \rightarrow I3$  (x3),  $I2 \rightarrow I4$  (x3); Control hazard: I4 (BEQ)。无 forwarding 时所有 RAW 都需要 stall；无分支预测则 BEQ 须等 EX 阶段算出结果，fetch 阻塞。[lec-02]

2. 下列哪种硬件结构**不会** typically 和动态调度 (dynamic scheduling) 联系在一起?
- A. Branch Delay Slots
  - B. Reservation Station
  - C. ROB (Reorder Buffer)
  - D. Register Rename Table

**解：**A。Branch Delay Slot 是静态调度技术 (编译器填充)，Reservation Station、ROB、Register Rename Table 三者是 Tomasulo 动态调度的核心硬件。[lec-03]

3. 有关记分牌 (Scoreboard) 算法和 Tomasulo 方法的区别，以下说法正确的是?

**解：**Scoreboard 只做动态调度但不消除 WAR/WAW (靠 stall 等待)，Tomasulo 用寄存器重命名消除 WAR/WAW，两者都处理 RAW。Tomasulo 将 Scoreboard 的控制集中式改为 Reservation Station 分布式。且 Tomasulo 配合 ROB 支持 Hardware Speculation，Scoreboard 不支持。[lec-03]

4. Cache 大小为 16 KB，Block Size 为 64 B，缓存是四路组相联的，求 set 数量。

**解：** $\#blocks = 16 \times 1024 / 64 = 256$ ,  $\#sets = 256 / 4 = 64$  个 set。[lec-04,05]

5. 对于直接映射 (DM)、全相联 (FA)、组相联 (SA) 缓存的 Miss Rate 而言 (假设容量和 Block Size 都一样):

- A. DM always 有更低的 Miss rate
- B. FA usually 有更低的 Miss rate
- C. SA always 比 DM 有更高的 Miss rate
- D. 由于 Size & Block Size 一样, 三者的 Cache Miss rate 相等

**解:** B. FA 无冲突缺失 (conflict miss), 在同等容量下通常 Miss Rate 最低。DM 冲突缺失最多。SA 介于两者之间。A/C 错在 “always”; D 忽略了映射方式对冲突缺失的影响。[lec-04,05]

6. 4 block、2-way set associative Cache, block size = 1 word, 采用写回 + 写分配策略, LRU 替换。对以下访问序列求 Cache miss 数量:

Read 0, Read 4, Read 0, Read 8, Read 4, Write 12, Write 0

**解:** 4 blocks, 2-way SA  $\rightarrow$  2 sets (set index = address mod 2)。

| Access | Set | Hit/Miss      | Set state (LRU left) |
|--------|-----|---------------|----------------------|
| R0     | 0   | Miss          | [0, _]               |
| R4     | 0   | Miss          | [0, 4]               |
| R0     | 0   | Hit           | [4, 0]               |
| R8     | 0   | Miss, evict 4 | [0, 8]               |
| R4     | 0   | Miss, evict 0 | [8, 4]               |
| W12    | 0   | Miss, evict 8 | [4, 12]              |
| W0     | 0   | Miss, evict 4 | [12, 0]              |

**总 miss = 6** (只有第 3 次 R0 命中)。[lec-04,05]

7. 有关 Vector Processors 的概念判断: 以下说法正确的是?

**解:** 向量处理器是 SIMD 架构的实现: 一条向量指令同时对一组向量元素做相同操作。向量元素之间很少有相关性, 所以向量流水线效率很高。向量处理器  $\neq$  *Scalar Processor* [lec - 18]

8. 在缓存全命中情况下 CPI = 1.0, Load/Store 占总指令 50%, Miss Rate = 2%, Miss Penalty = 20 时钟周期。求在保证全部缓存命中下的 Speedup。

**解:** Memory stall cycles =  $IC \times 0.5 \times 0.02 \times 20 = 0.2 IC$ 。

实际 CPI:  $CPI_{actual} = 1.0 + 0.2 = 1.2$ 。

全部命中 CPI:  $CPI_{ideal} = 1.0$ 。

Speedup =  $1.2/1.0 = 1.2$ 。[lec-07]

9. SCAN 算法 (电梯算法) 读下列 Cylinder (假设当前磁头在 50, 向大号方向移动, 请求队列: 95, 180, 34, 119, 11, 123, 62, 64), 求磁头移动总路程。

**解：**SCAN 从 50 出发向右到 180，再折返向左到 11：50 → 62 → 64 → 95 → 119 → 123 → 180 → 34 → 11。

路程 = (62-50)+(64-62)+(95-64)+(119-95)+(123-119)+(180-123)+(180-34)+(34-11) = 12 + 2 + 31 + 24 + 4 + 57 + 146 + 23 = **299**。[lec-13]

10. (页表相关) 某系统采用 32-bit 虚拟地址、4KB 页面大小，两级页表 (每级 1024 个 PTE, 每个 PTE 4B)。求：每级页表大小，以及虚拟地址 VPN 各段的位数划分。

**解：**4KB 页 → offset 12 bit。两级各 1024 项 → 每级 10 bit。每级页表大小 = 1024 × 4 = 4KB，正好一页。划分：[VPN1 10 bit | VPN0 10 bit | offset 12 bit]。[lec-09]

11. (VM 相关) 虚拟内存和物理内存的关系以下哪项正确？

- A. 虚拟地址空间不能大于物理地址空间
- B. 虚拟内存存在物理内存不足时仍可让程序运行
- C. 每个进程共享同一张页表
- D. 虚拟内存就是交换空间 (swap)

**解：**B。虚拟地址空间可以远超物理内存 (按需调页)，每个进程有独立页表，交换空间只是虚拟内存的实现手段之一，不是等价概念。[lec-10]

12. RISC-V 架构下的 PTE 不包含下列哪个元素？

- A. Logic Page Number
- B. Physical Page Frame Number
- C. Page Residence (Valid) bit
- D. Page Permissions

**解：**A。PTE 存的是 PPN (物理页框号)、valid/dirty/accessed 等标志位和 R/W/X 权限位。VPN (逻辑页号) **不在 PTE 里**——它作为 offset 在页表中索引 PTE，不存储在 PTE 内部。[lec-09,11]

13. 使用 LRU 替换策略，页框初始为空。对以下引用串求 Page Fault 次数 (假设分配 3 个页框)：

2 3 2 1 5 2 4 5 3 2 5 2

| 访问   | 页框      | PF?         | 备注 |
|------|---------|-------------|----|
| 2    | [2,_,_] | PF=1        |    |
| 3    | [2,3,_] | PF=2        |    |
| 2    | [2,3,_] | hit         |    |
| 1    | [2,3,1] | PF=3        |    |
| 5    | [5,3,1] | PF=4, LRU=2 |    |
| 解: 2 | [5,2,1] | PF=5, LRU=3 |    |
| 4    | [5,2,4] | PF=6, LRU=1 |    |
| 5    | [5,2,4] | hit         |    |
| 3    | [3,2,5] | PF=7, LRU=4 |    |
| 2    | [3,2,5] | hit         |    |
| 5    | [3,2,5] | hit         |    |
| 2    | [3,2,5] | hit         |    |

总 Page Fault = 7。[lec-10]

14. (分支预测/猜测执行相关) 在 Hardware-Based Speculation 中, 以下说法 **错误**的是?

- A. ROB 在 Commit 阶段才将结果写回寄存器文件
- B. 分支预测失败时, ROB 中错误路径的指令会被 flush
- C. Speculation 允许指令在分支结果确认前执行
- D. Write Result 阶段直接将结果写入寄存器文件

**解:** D。在 Hardware Speculation 中, Write Result 阶段将结果写入 ROB (而非直接写寄存器文件), Commit 阶段才从 ROB 写回寄存器文件。这保证了错误的推测结果不会污染体系结构状态。  
[lec-03]

15. (综合判断) 关于 DLP 和 TLP 的对比, 以下说法正确的是?

- A. SIMD 属于 TLP 的实现
- B. MIMD 利用 TLP, 适合并行执行多个线程
- C. Flynn 分类中, SISD 包含向量处理器
- D. DLP 通过提高时钟频率来加速程序

**解:** B。SIMD 属于 DLP (数据级并行), 不是 TLP。MIMD 利用 TLP 让多个独立线程在不同处理器/核上同时运行。向量处理器是 SIMD 不是 SISD。DLP 一次操作多个数据, 不靠提频。[lec-18,19]

## 二、大题

### 1. Hardware-Based Speculation (15')

设仅有一个 Reservation Station (RS) + ROB。加/减法 1 周期, 乘法 6 周期, 除法 20 周期。  
填写下表 (所有指令在 ROB 中按序提交):

| inst | operands | issue | execute | write result | commit |
|------|----------|-------|---------|--------------|--------|
| div  | x2,x3,x4 | x3,x4 |         |              |        |
| mul  | x1,x5,x6 | x5,x6 |         |              |        |
| add  | x3,x7,x8 | x7,x8 |         |              |        |
| mul  | x1,x1,x3 | x1,x3 |         |              |        |
| sub  | x4,x1,x5 | x1,x5 |         |              |        |

**解：**仅 1 个 RS → 指令须串行发射（前一条 WB 释放 RS 后才发下一条）。ROB 顺序：div(1) → mul(2) → add(3) → mul(4) → sub(5)，按序提交。

| inst | operands | issue | execute | write result | commit |
|------|----------|-------|---------|--------------|--------|
| div  | x3,x4    | 1     | 1–20    | 21           | 21     |
| mul  | x5,x6    | 22    | 22–27   | 28           | 28     |
| add  | x7,x8    | 29    | 29      | 30           | 30     |
| mul  | x1,x3    | 31    | 31–36   | 37           | 37     |
| sub  | x1,x5    | 38    | 38      | 39           | 39     |

说明：

- div issue@c1, EX 1–20, WB@c21, RS 释放；
- mul x1,x5,x6 issue@c22 (RS 空, x5/x6 就绪), EX 22–27, WB@c28；
- add issue@c29, EX 1 周期, WB@c30；
- 第二条 mul 的 x1 来自第一条 mul (WB@c28), x3 来自 add (WB@c30), 最早 issue@c31, EX 31–36, WB@c37；
- sub issue@c38 (等 mul WB@c37 释放 RS), EX 1 周期, WB@c39。

注：单 RS 是**严重瓶颈**，真实 Tomasulo 每种 FU 有独立 RS 可并行发射。本题令 RS 串行化是故意考察对 issue 条件的理解。 [lec-03]

## 2. MSI Cache Coherence Protocol (15')

图与示例与课堂 PPT 相同（三处理器 C0, C1, C2，总线 snooping）。

(1) C0: W, A11C  $\leftarrow$  012C

(2) C1: W, A108  $\leftarrow$  0308

(3) C2: R, A108

**解：**MSI 三状态：Modified（脏，独占）、Shared（干净，可能多副本）、Invalid。

- (1) **C0 写 A11C**：C0 发 BusRdX 获取 A11C 的独占副本，其他缓存中该行标 I。C0 拿到后写入 012C，状态 → M。
- (2) **C1 写 A108**：C1 发 BusRdX，若 C0 持 M 态则需要写回（write-back）并标 I。C1 拿到独占副本，写入 0308，状态 → M。
- (3) **C2 读 A108**：C1 (M 态) snoop 到 BusRd，将数据写回内存，C1 → S，C2 从总线拿数据 → S（共享）。

关键：M 态被 snoop 命中时必须 flush 到内存再转 S；所有状态转换由总线事务触发。 [lec-3-4?] !!!

课件缺 MSI 内容，需自行补充

### 3. RISC-V Sv39 虚拟内存 (20')

RISC-V Sv39 地址翻译图 (与 Lab 3 实验指导中相同)。

- (1) 分别阐述 segmentation 与 paging 各自的优劣势。
- (2) RISC-V 64 是否支持 segmentation?

给定代码:

```
int* buf = NULL;
int main() {
    buf = (int*)malloc(4096 * 16);
    buf[0] = 1919;
}
```

- (3)
  - (i) malloc 一句是否分配了物理内存?
  - (ii) 哪行触发了 page fault?
  - (iii) 整段代码运行时一共分配了多少物理空间?
- (4)
  - (i) 整段代码运行过程中一共触发了几次 page fault?
  - (ii) page fault 分别是由哪些页导致的?
- (5) 写出虚拟地址 0xc0000000 → 物理地址 0xb000 的完整地址翻译过程 (Sv39 三级页表 walk)。

解:

- (1)
  - **Segmentation 优势**: 无内部碎片, 天然对应逻辑单元 (代码/数据/栈), 便于共享和保护。
  - **Segmentation 劣势**: 外部碎片严重, 分配复杂 (best-fit/first-fit), 需要紧凑 (compaction)。
  - **Paging 优势**: 无外部碎片, 分配简单 (固定大小 frame), 支持虚拟内存 (demand paging)。
  - **Paging 劣势**: 内部碎片 (平均每进程  $\frac{1}{2}$  页), 页表存储开销大, 地址翻译多级访问内存。
- (2) RISC-V 64 **不支持** segmentation。RISC-V 的特权架构采用纯分页模型 (Sv39/Sv48/Sv57), 虚拟地址到物理地址通过多级页表翻译, 无段寄存器或段描述符机制。[lec-08,09]
- (3)
  - (i) **没有**。malloc 仅在堆的虚拟地址空间中**预留**了  $4096 \times 16 = 64\text{KB}$  (16 页), 但未分配物理页框。物理内存采用按需调页 (demand paging), 实际分配发生在首次访问时。
  - (ii) buf[0] = 1919 触发了 page fault。这是对刚 malloc 的虚拟地址区域的**首次写访问**。
  - (iii) 仅分配了 **1 页 = 4KB** 物理内存。只有 buf[0] 所在的那一页被访问到, 其他 15 页从未被触碰。
- (4)
  - (i) 共触发 **1 次** page fault (buf[0] = 1919 那一次)。程序的代码段和栈在进程启动时已被 OS 预加载 (pre-fault)。
  - (ii) 由 buf 指向的堆区域第 0 页导致 (该页在 malloc 返回的虚拟地址范围内的第一页)。

(5) **Sv39 三级页表翻译:**

Sv39: 虚拟地址 39 bit, 页大小 4 KB  $\rightarrow$  offset 12 bit, 三级页表每级 9 bit (VPN[2] : VPN[1] : VPN[0] : offset = 9 : 9 : 9 : 12)。

0xc0000000 二进制: 1100 0000 0000 0000 0000 0000 0000 0000

- VPN[2] (38:30) = 0b110000000 = **0x180**
- VPN[1] (29:21) = 0b000000000 = **0x000**
- VPN[0] (20:12) = 0b000000000 = **0x000**
- Offset (11:0) = 0x000

翻译步骤:

- (a) 读 **satp** 寄存器获取根页表 PPN;
- (b) L2: 读 PTE[root + 0x180], 得 L1 页表 PPN;
- (c) L1: 读 PTE[L1 + 0x000], 得 L0 页表 PPN;
- (d) L0: 读 PTE[L0 + 0x000], 得最终 PPN = 0xb;
- (e) PA = 0xb << 12 | 0x000 = **0xb000**。

每次 PTE 访问都可能触发 page fault (若 PTE 无效或非叶节点), 最坏需 3 次内存访问才完成 walk; TLB 命中则只需 1 次。[lec-09,10,11]

#### 4. File System (20')

- (a) 采用 indexed disk block allocation (设 block size = 4 KB, 指针 4 B), 以下各需几个 blocks (含 index block)?
  - (i) 70 B
  - (ii)  $2^{20} + 2044$  B
  - (iii)  $2^{20} + 2044$  B (与 (ii) 相同, 按原题保留)
- (b) Indexed disk block allocation 能分配的最大空间 (单 index block)。
- (c) 假设最多使用 3 个 disk blocks 存放索引结构, 求能分配的最大空间。
- (d) 以下信息保存在 directory 中还是 inode 中, why?
  - (i) time of creation
  - (ii) file name
  - (iii) access rights
- (e) Comment the performance of **contiguous blocks** on reading, writing and creating files.

**解:**

- (a) 每 index block 可存  $4096/4 = 1024$  个数据块指针。
  - (i) 70 B: 1 数据块 ( $70 \text{ B} < 4096 \text{ B}$ )。总 = **1 index + 1 data = 2 blocks**。

(ii)  $2^{20} + 2044 = 1\,048\,576 + 2044 = 1\,050\,620\text{ B}$ 。  $\lceil 1\,050\,620/4096 \rceil = 257$  数据块。单 index block (1024 指针) 足够。总 = **1 index + 257 data = 258 blocks**。

(iii) 同上。(原卷回忆二、三问数字不同, 此处按原文保留)

(b) 单 index block: 1024 指针, 每数据块 4KB。  $1024 \times 4\text{ KB} = \mathbf{4\text{ MB}}$ 。 [lec-14,15]

(c) 若支持多级索引: 3 个 index blocks 可构成

- 1 个一级 index block  $\rightarrow$  指向 1024 数据块;
- 1 个二级 index block  $\rightarrow$  指向 1024 个一级 index block  $\rightarrow 1024^2$  数据块;
- 1 个三级 index block  $\rightarrow 1024^3$  数据块。

极端情况下 (三级间接 + 两级间接 + 一级间接),  $\approx 1024^3 \times 4\text{ KB} \approx \mathbf{4\text{ TB}}$ 。

若仅为 3 个平级 index block:  $3 \times 1024 \times 4\text{ KB} = \mathbf{12\text{ MB}}$ 。

(两种理解均可, 取决于是否允许多级索引结构。) [lec-15,16]

(d) (i) **time of creation**  $\rightarrow$  **inode**。创建时间是文件元数据 (metadata), UNIX 系统存于 inode 中 (如 `st_ctime`)。

(ii) **file name**  $\rightarrow$  **directory**。Directory 的核心功能就是从人类可读的名字  $\rightarrow$  inode 号的映射。inode 本身不存文件名 (一个文件可以有多个硬链接, 即多个名字)。

(iii) **access rights**  $\rightarrow$  **inode**。权限位 (r/w/x) 及 owner/group 存于 inode 的 **mode** 字段。

[lec-14,16]

(e) **Contiguous allocation** 性能分析:

- **Reading**: 极优。文件在磁盘上连续存放, 顺序读只需一次 seek + 旋转延迟后可持续传输, I/O 带宽利用率最高。
- **Writing**: 顺序写极优 (追加); 随机写若需扩展文件则可能需整体搬迁。
- **Creating**: 困难。需在磁盘上找到足够大的连续空闲区域 (类似内存的 external fragmentation 问题)。随使用时间增长, 空闲空间碎片化, 大文件创建越来越难。

综合评价: contiguous 在读/写吞吐量上最优, 但空间管理代价最高。现代文件系统 (ext4, NTFS) 通常混合使用 extent-based 分配 + 按需碎片整理, 而非纯 contiguous。 [lec-14,15]